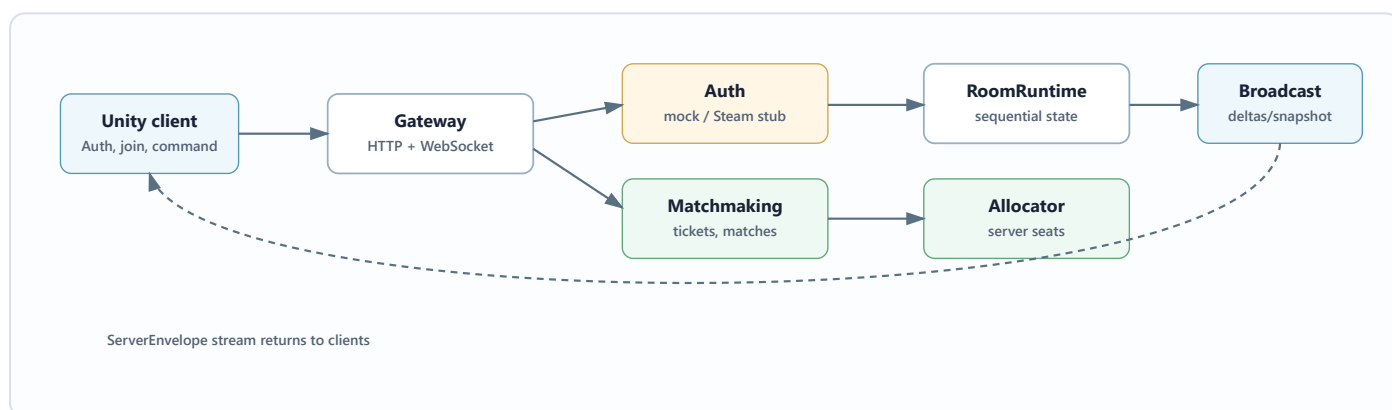


# Как работает Ruleshift

Быстрое объяснение архитектуры Go-сервера: от matchmaking ticket до assignment, от WebSocket-команды до авторитетного изменения состояния комнаты, от backpressure до recovery через snapshot.

**ЦЕЛЬ****Multiplayer backend****ТЕКУЩИЙ MVP****shared int64 state****ТРАНСПОРТ****WebSocket +  
protobuf****ДАТА****04.06.2026**

Ruleshift демонстрирует не CRUD, а авторитетный real-time pipeline: клиенты отправляют intent, сервер упорядочивает команды, меняет состояние и рассылает всем игрокам один и тот же поток ревизий.



1. Что уже реализовано и где это лежит

2. Поток ticket -> assignment

3. Как работает авторитетная комната

4. Backpressure, reconnect и replay

5. Надежность, TTL, idempotency и failure path

6. Производительность, проверки и следующие шаги

# 1. Короткая карта системы

Ruleshift сейчас состоит из двух уровней. Первый уровень уже подключен к публичному gateway: клиент проходит auth, подключается к room и шлет protobuf-команды изменения одного общего [int64](#). Второй уровень появился как внутренний orchestration layer: он моделирует будущий сервис matchmaking и assignment, но пока не вынесен наружу как HTTP или protobuf API.

| ПАКЕТ                              | ОТВЕТСТВЕННОСТЬ                                                                | ЧТО ВАЖНО ПОМНИТЬ                                             |
|------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------|
| <code>cmd/gateway</code>           | Собирает HTTP server, конфиг, health/readiness/metrics, graceful shutdown.     | Здесь не должно быть domain logic.                            |
| <code>internal/gateway</code>      | WebSocket state machine, auth-first protocol, protobuf decode/encode.          | Session outbound queue bounded, writer loop вне room runtime. |
| <code>internal/room</code>         | Авторитетное состояние комнаты, последовательное применение команд, broadcast. | Одна room - один actor-like runtime и bounded input queue.    |
| <code>internal/matchmaking</code>  | Ticket lifecycle, match formation, assignment lifecycle, события.              | In-memory слой, готовый для будущих API handlers.             |
| <code>internal/allocator</code>    | Регистрация серверов и атомарное резервирование seats.                         | Reserve/release под одним mutex, индекс по game/build.        |
| <code>internal/connecttoken</code> | HMAC-signed connect tokens для game server admission.                          | Токен содержит assignment/match/server/player/expiry claims.  |
| <code>internal/protocol</code>     | Protobuf schema, generated bindings, codec helpers, protocol version.          | FlatBuffers в MVP не используются.                            |

### Что уже можно показать

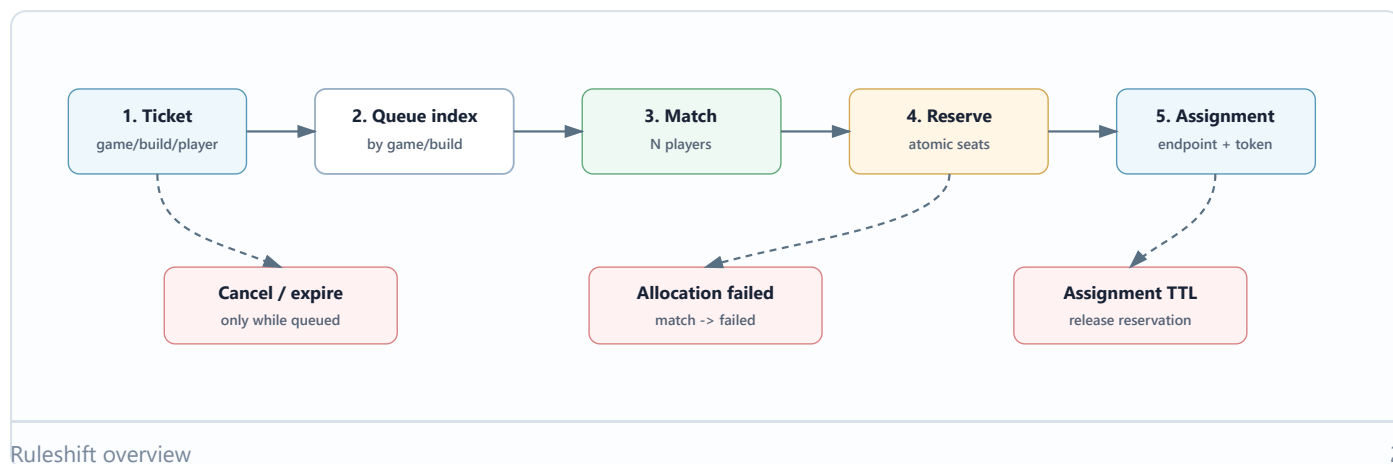
- Авторитетный ordered revision stream для общей integer-room.
- Reconnect через `last_seen_revision` и snapshot recovery.
- Slow consumer handling: compact to snapshot, затем disconnect.
- Matchmaking lifecycle в unit-tested internal layer.
- Atomic allocation без двойного назначения одного seat.

### Что пока не публичный продукт

- Нет внешнего API для create/cancel ticket и poll assignment.
- Нет durable storage: текущие stores in-memory.
- Нет настоящего launcher для game server build.
- Steam provider пока skeleton, mock auth используется локально.

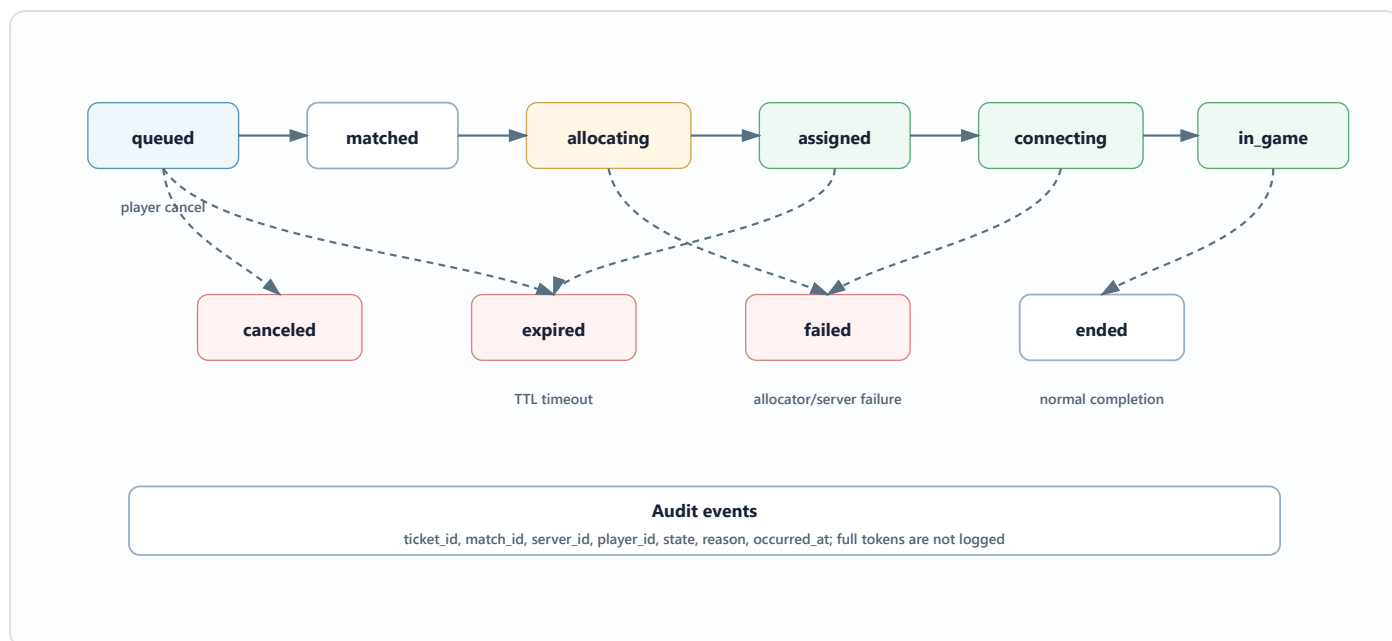
## 2. Поток от ticket до assignment

Целевой сервис начинается не с комнаты, а с matchmaking ticket. Игрок хочет попасть в игру, сервис группирует совместимые tickets, резервирует серверные seats и выдает assignment: endpoint, match id, server id и connect token.



### 3. Lifecycle state machine

Главное правило lifecycle: каждый переход должен быть допустимым и объяснимым. Это важно для диагностики: разработчик игры должен видеть, где матч умер - в queue, allocation, assignment TTL, connect validation или уже внутри игры.



#### Idempotency

Повторный create-ticket с тем же idempotency key возвращает тот же ticket. Активный ticket игрока в том же game/build pool тоже переиспользуется.

#### Atomic allocation

Registry резервирует seats под mutex и индексирует servers по game/build. Один seat не может быть выдан двум matches.

#### TTL cleanup

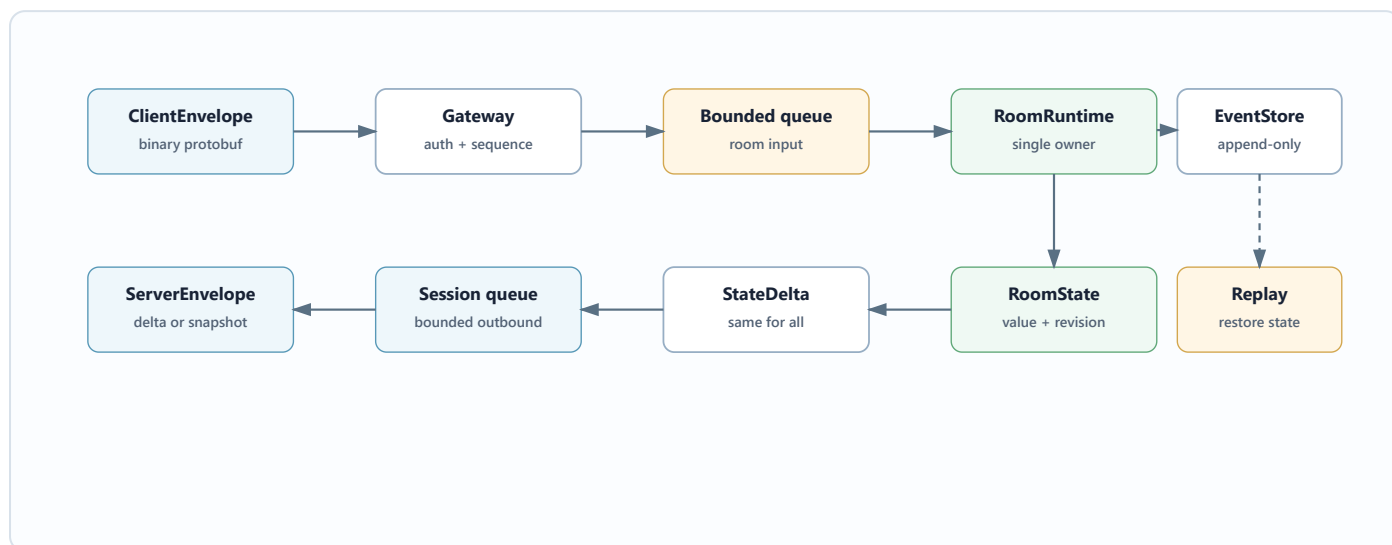
Queued ticket может expire. Assignment TTL failure переводит match в failed и освобождает server reservation.

### 4. Как game server проверяет игрока

Connect token - это HMAC-signed claim set. В нем есть `assignment_id`, `match_id`, `server_id`, `player_id` и `expires_at`. Game server принимает игрока только если token валиден, не истек и claims совпадают с assignment. В логах используется идентификатор assignment, но не полный token.

## 5. WebSocket room pipeline

После того как клиент получил право играть, текущий публичный MVP работает через WebSocket endpoint `/ws`. Клиент отправляет binary protobuf `ClientEnvelope`. Gateway проверяет protocol version, sequence и session state, затем отправляет команду в room runtime.



### Почему runtime последовательный

Команды всех клиентов попадают в bounded room input queue. Один runtime goroutine владеет `RoomState`, поэтому состояние не мутируется конкурентно. Принятая команда увеличивает revision ровно на один.

### Почему клиенты видят один порядок

Runtime строит один `StateDelta` после применения команды и отправляет тот же envelope всем joined sessions. Клиент не сообщает новый state, он сообщает только intent.

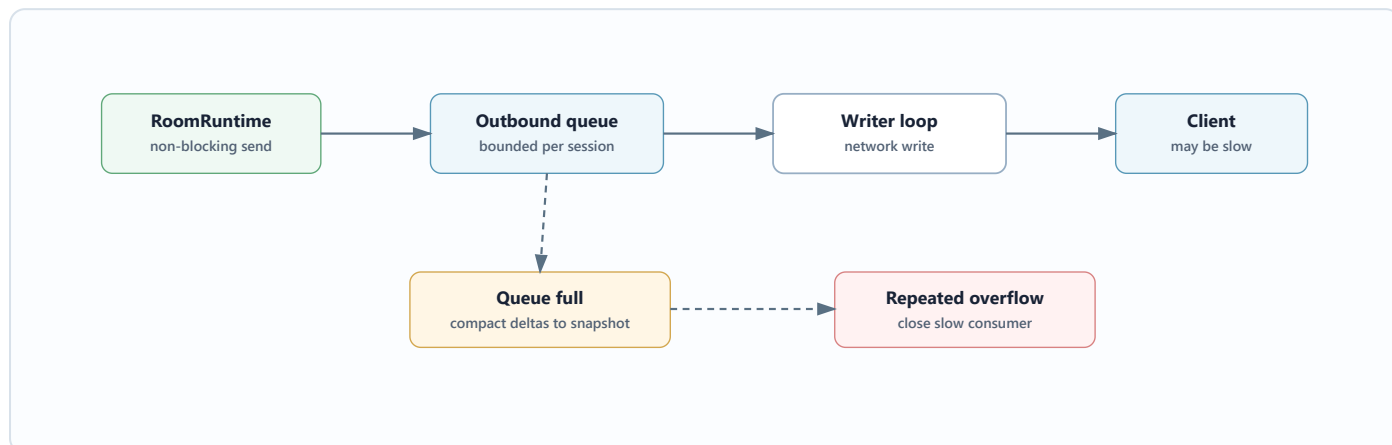
## 6. Reconnect, snapshot и replay

Клиент хранит последнюю примененную revision. При `JoinRoomRequest` сервер сравнивает `last_seen_revision` с текущей revision комнаты. Если они равны, достаточно `JoinRoomOk`. Если нет - сервер отправляет `StateSnapshot`.

Event log нужен для восстановления state из server-side истории: `RoomCreated`, затем `IntAdded` и `IntSet`. Join, snapshot sent и disconnect остаются audit events, но не меняют integer value.

## 7. Backpressure и slow consumers

Hot path не должен блокироваться на сетевые записи. Поэтому gateway session имеет bounded outbound queue, а WebSocket writer loop дренирует ее отдельно от room runtime. Если клиент читает медленно, runtime сначала пытается заменить накопившиеся deltas свежим snapshot. Если проблема повторяется, session закрывается как `slow_consumer`.



### Что это защищает

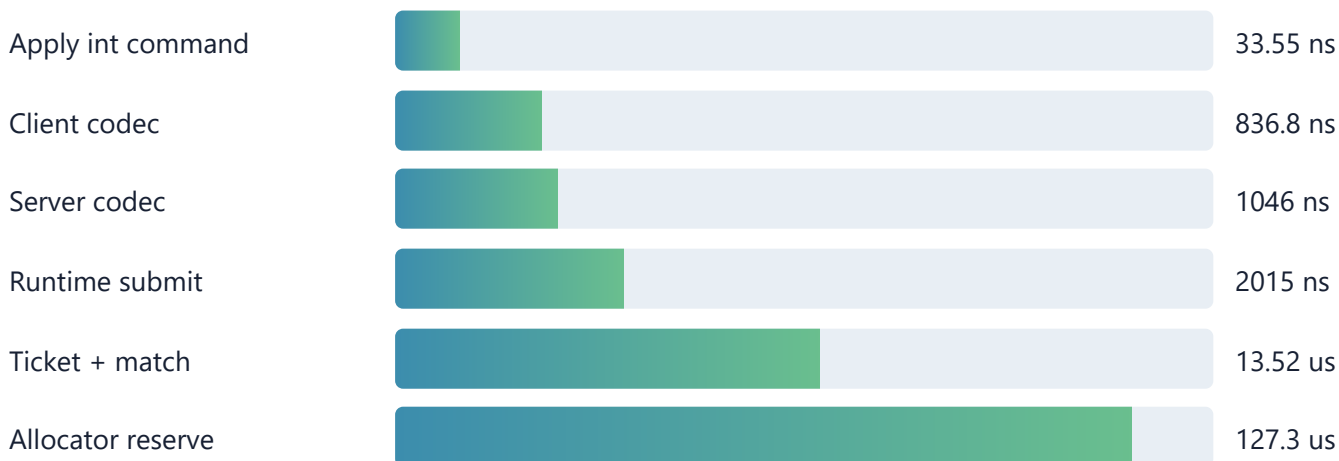
- Room runtime не зависает на одном плохом socket.
- Очереди bounded, значит нет бесконечного роста памяти.
- Snapshot recovery дает клиенту компактную актуальную картину state.

### Что можно улучшить дальше

- Добавить ring buffer deltas для catch-up без full snapshot.
- Вынести Prometheus counters для queue full и disconnect reasons.
- Сделать registry cleanup закрытых или пустых rooms.

## 8. Производительность: текущие ориентиры

Ниже не SLA, а quick orientation по последнему benchmark-прогону на Windows amd64, Intel i5-9400F. Важен порядок величин: command reducer без аллокаций, protobuf codec в микросекундной зоне, orchestration layer дороже из-за maps, ids и lifecycle events.



## 9. Как быстро разобраться в коде

Если открыть проект впервые, лучше читать не сверху вниз по директориям, а по двум потокам: "игрок подключается к room" и "игрок получает assignment". Так проще увидеть границы пакетов.

### Поток room MVP

1. `cmd/gateway/main.go` - HTTP wiring, config, shutdown.
2. `internal/gateway/gateway.go` - WebSocket protocol state machine.
3. `internal/gateway/session.go` - bounded outbound session queue.
4. `internal/room/runtime.go` - actor-like room runtime.
5. `internal/room/command.go` - reducer for Add/Set.
6. `internal/protocol/proto/ruleshift.proto` - wire contract.

### Поток matchmaking

1. `internal/matchmaking/service.go` - ticket/match/assignment lifecycle.
2. `internal/allocator/allocator.go` - server pool and seat reservation.
3. `internal/connecttoken/token.go` - signed token generation/validation.
4. `internal/matchmaking/service_test.go` - cancellation, failure, TTL, concurrent assign.
5. `internal/allocator/allocator_test.go` - atomic reservation and cleanup.

## 10. Operational surface

| ENDPOINT / CONFIG                              | ЗАЧЕМ НУЖЕН                                                                           |
|------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>/healthz</code>                          | Простой liveness check: процесс отвечает.                                             |
| <code>/readyz</code>                           | Readiness check: gateway готов принимать запросы.                                     |
| <code>/metrics</code>                          | Минимальный text endpoint: <code>ruleshift_up</code> и <code>ruleshift_rooms</code> . |
| <code>/ws</code>                               | WebSocket endpoint для protobuf room protocol.                                        |
| <code>RULESHIFT_ROOM_INPUT_QUEUE_SIZE</code>   | Размер bounded queue команд на комнату.                                               |
| <code>RULESHIFT_SESSION_SEND_QUEUE_SIZE</code> | Размер bounded outbound queue на WebSocket session.                                   |
| <code>RULESHIFT_SHUTDOWN_TIMEOUT</code>        | Deadline graceful shutdown для HTTP server и gateway close.                           |

## 11. Что проверено тестами

### ROOM

linear revisions, broadcasts, invalid commands, slow consumers, shutdown

Ruleshift overview

### GATEWAY

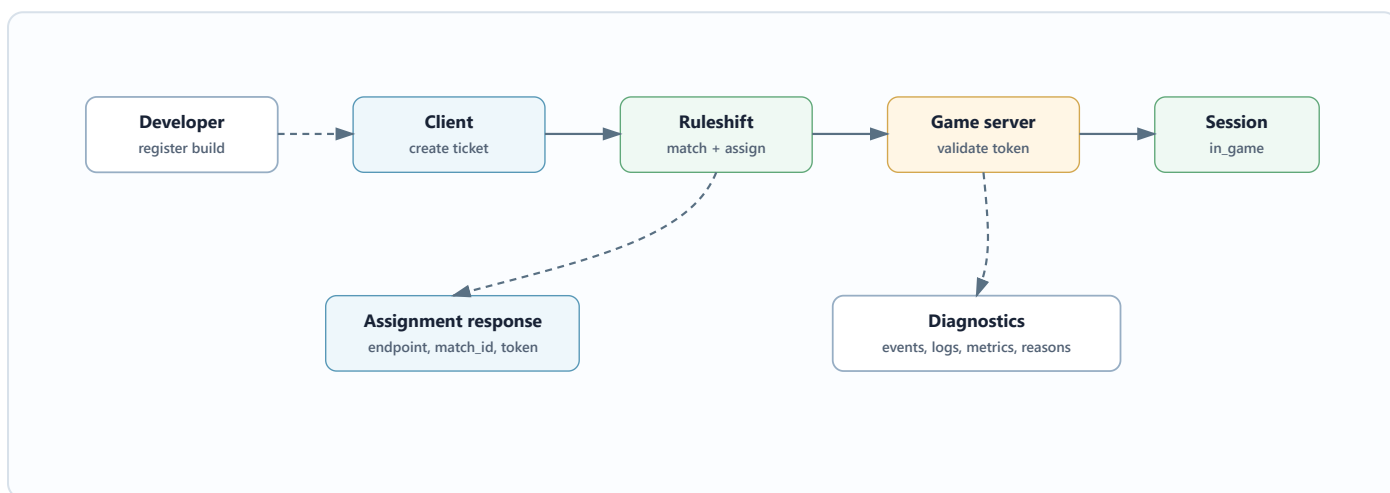
auth-first flow, join, snapshots, reconnect, ping/pong

### ORCHESTRATION

ticket cancel, match, allocation, TTL, token, failure, concurrency

## 12. Как выглядит полный будущий flow

1. Разработчик регистрирует игру и build. Пока durable registry не реализован, allocator получает server records in-memory.
2. Клиент создает ticket: `game_id`, `build_id`, `player_id`, optional idempotency key.
3. Matcher группирует queued tickets одного game/build pool в match.
4. Allocator атомарно резервирует seats на подходящем server instance.
5. Сервис создает assignment для каждого игрока и выдает endpoint + connect token.
6. Game server проверяет token через token manager или будущий validation endpoint.
7. После принятия игрока lifecycle идет в `connecting`, затем `in_game`.
8. Когда игра заканчивается, reservation освобождается, match становится `ended`.



## 13. Следующие PR

### Самый полезный следующий шаг

Подключить public API: create/cancel ticket, poll assignment, validate token, game server heartbeat, mark in\_game, end/fail session. Это превратит внутренний lifecycle layer в настоящий продуктовый контур.

### После API

Добавить durable storage, real server launcher, Prometheus metrics, pprof, load test execution и Steam auth implementation. Тогда система будет готова к более честной production-like демонстрации.

Итог: текущий Ruleshift уже показывает ключевые системные идеи: authoritative state, bounded queues, deterministic ordering, recovery snapshots, lifecycle audit и atomic allocation. Главная недостающая часть - публичный API и durability вокруг нового orchestration layer.